

CS1910

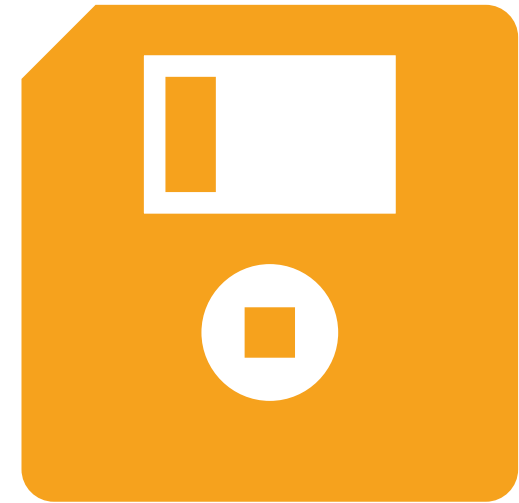
Files and Folders

Reading from Files

Writing to Files

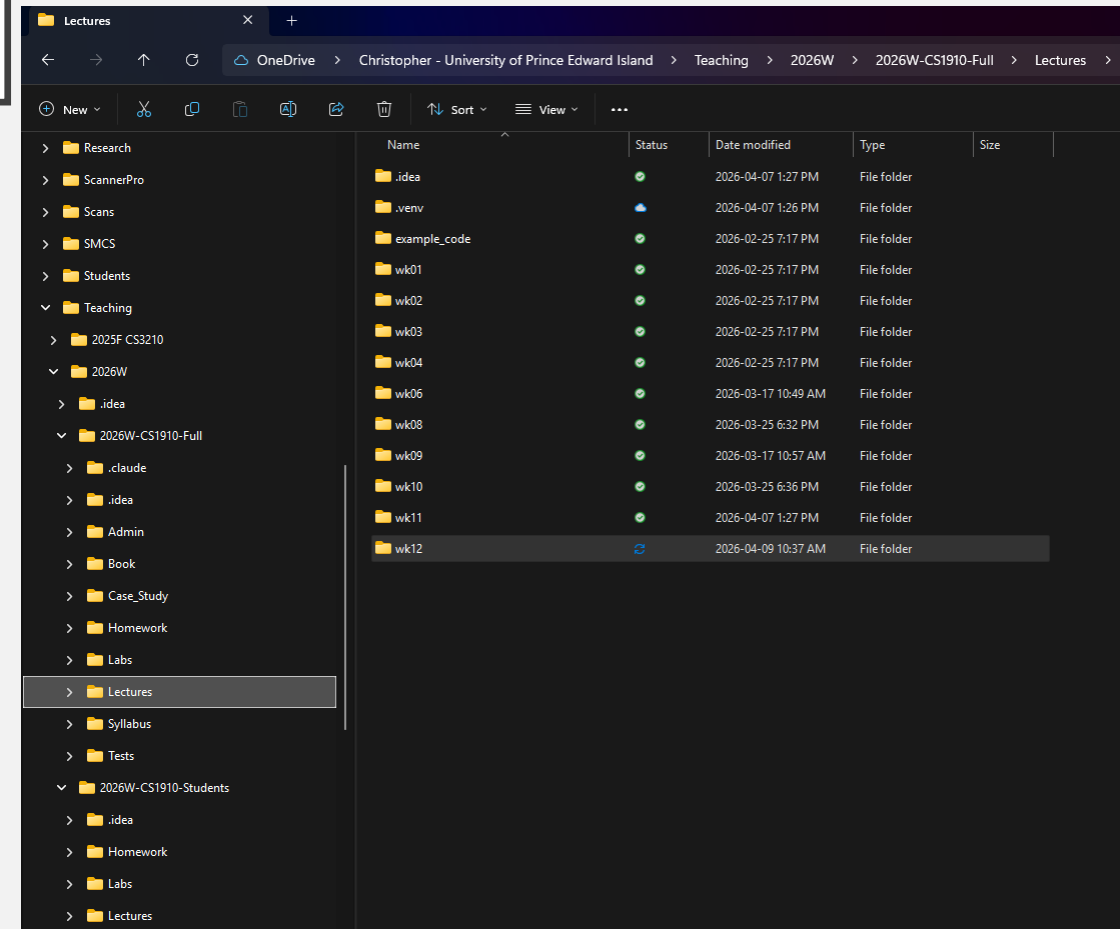
TYPES OF STORAGE

- Program data when running is in memory only when power is running
- Programs themselves need to be stored for long term use even when power is turned off
- Internal Storage Devices:
 - Hard drive
 - Originally tape, then spinning platters of discs, increasingly are solid state drives (SSD)
- External Storage Devices
 - External drives that are connected via USB or other connector
 - USB sticks – smaller versions of SSD that can be removed
 - Increasingly – cloud services
- Can hold: text, images, videos, programs, binary information



FILE SYSTEMS

- File – a structured collection of data that is grouped under an abstraction of a name with an “extension” that tells you the type of data that is stored within
- Folder – an abstraction that gives a name that collects files together that are related to one another (previously called “directory”)
- The metaphor is that of folders in an office drawer that hold files of paper – like a case file in a lawyer’s office
- Where it breaks down as a metaphor is:
 - Files can be put in folders
 - Folders can be put in other folders creating hierarchies
- A unique location of a file is defined by it’s path:
 - \Teaching\2026W\2026W-CS1910-Full\Lectures\wk12\Lecture-FileIO.pptx (Windows)
 - \Teaching\2026W\2026W-CS1910-Student\Lectures\wk12\Lecture-FileIO.pptx (Windows)
 - /Teaching/2026W/2026W-CS1910-Student/Lectures/wk12/Lecture-FileIO.pptx (Mac, Linux)



DATA FILE FORMATS

- Two main file formats:
 - Text
 - Readable by humans
 - Everything is a string (even numbers)
 - Binary
 - Generally not readable by humans
 - The data is binary encoded stored in groups of bytes (8 bits)
 - Binary is 0-1 encoding where each 0 or 1 is called a bit
 - Offer some efficiencies in storage
 - Not the focus in this course (aside from understanding they exist)
 - Examples: images, songs, programs

TEXT FILES

- Some common Text files:
 - Plain text: .txt, .md
 - Source code: .py,
 - Structured Data: .csv, .ipynb
 - hypertext: .html

You can open any of these files with a “text editing” program and read the contents

.IPYNB

Even our Notebook files are
plain text

That is JSON (the syntax is
similar to a python dictionary)

Opening HW7.ipynb with a text editor reveals:

```
{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {
        "id": "view-in-github",
        "colab_type": "text"
      },
      "source": [
        "<a href='\"https://colab.research.google.com/github/andrewgodbout/F2021/\"'>Open In Colab</a>"
      ]
    },
    {
      "cell_type": "markdown",
      "id": "b41221c2-9cc9-42b6-87a1-51d79d8d0373",
      "metadata": {
        "id": "b41221c2-9cc9-42b6-87a1-51d79d8d0373"
      },
      "source": [
        "## Before you begin\n",
        "\n",
        "### Submission\n",
        "Before you start working on this Homework:\n",
        "\n",
        "1. Press **File->Save a copy in Drive** from the menu. This will save\n",
        "\n",
        "2. Follow the steps below and ensure you run the cells in order from 1\n",
        "\n",
        "3. You should get into the habit of saving your file occassionally as\n",
        "\n",
        "4. Certain cells will check your work against the expected answer, wh\n",
        "\n",
        "5. After all the changes are done and progress is saved **File->Downl"
      ]
    }
  ],
  "metadata": {
    "id": "b41221c2-9cc9-42b6-87a1-51d79d8d0373"
  }
}
```

OPENING AND CLOSING A FILE IN PYTHON

f can be used to read the file

Filename to open

Flag to indicate read

```
f = open("temperatures.txt", "r")
```

temperatures.txt

```
-2.7  
-1.8  
0.3  
2.4  
3.5  
5.9
```

```
f.close()
```

Always close a file when you are done
(releases the resource)
f can no longer be used to read the file

contains one temperature
listed on each line of the file

READING FILES

- Read a file line by line

```
f = open("temperature.txt", "r")  
  
#reading a file line by line  
for line in f:  
    print(line)  
  
f.close()
```

output

-2.7
-1.8
0.3
2.4
3.5
5.9

temperatures.txt

-2.7
-1.8
0.3
2.4
3.5
5.9

READING FILES

- Read a file line by line

```
f = open("temperature.txt", "r")  
  
#reading a file line by line  
for line in f:  
    print(line)  
  
f.close()
```

output

-2.7
-1.8
0.3
2.4
3.5
5.9

Why the extra space
after each line?

temperatures.txt

-2.7
-1.8
0.3
2.4
3.5
5.9

READING FILES

read line by line and store into list

```
f = open("temperature.txt", "r")
```

```
lines = []  
for line in f:  
    lines.append(line)
```

```
f.close()  
print(lines)
```

```
['-2.7\n', '-1.8\n', '0.3\n', '2.4\n', '3.5\n', '5.9']
```

The file ends abruptly

The newline character is read with each line as well

READING FILES

```
# remove the newlines

f = open("temperature.txt", "r")

lines = []
for line in f:
    # strip away trailing (rightside) whitespace with rstrip
    lines.append(line.rstrip())

f.close()
print(lines)
```

```
['-2.7', '-1.8', '0.3', '2.4', '3.5', '5.9']
```

SELF-CHECK

temperatures.txt

-2.7
-1.8
0.3
2.4
3.5
5.9

```
#what is the output?  
f = open("temperature.txt", "r")  
  
cnt = 0  
for temp_line in f:  
    if cnt == 0:  
        mn_temp = temp_line  
    if temp_line < mn_temp:  
        mn_temp = temp_line  
    cnt += 1  
f.close()  
print(f"min: {mn_temp}")
```

SELF-CHECK

The lexicographically
ordered smallest line of
text from the file
i.e.,:“-1.8”

```
#what is the output?
f = open("temperature.txt", "r")

cnt = 0
for temp_line in f:
    if cnt == 0:
        mn_temp = temp_line
    if temp_line < mn_temp:
        mn_temp = temp_line
    cnt += 1
f.close()
print(f"min: {mn_temp}")
```

CONVERTING TEXT

```
f = open("temperature.txt", "r")

cnt = 0
for temp_line in f:
    temp_line = float(temp_line)
    if cnt == 0:
        mn_temp = temp_line
    if temp_line < mn_temp:
        mn_temp = temp_line
    cnt += 1
f.close()
print(f"min: {mn_temp}")
```

READING FILES

- An alternative to the repetition-based line reading

```
# Convert to list

f = open("temperature.txt", "r")

f_list = list(f)
f_list_float = [float(flt) for flt in f_list]

#print the min
print(min(f_list_float))
```

CONSIDER A TABULAR FILE

temptable.txt

1783	22	25	27	28	21	19
2214	-4	2	6	7	6	0
9934	-40	-32	-26	-21	-24	-32
5538	15	17	21	22	23	19

Weatherstation ID

Temperatures from that weatherstation

TABULAR FILE

```
# Tabular file  
  
f = open("temptable.txt", "r")  
for line in f:  
    print(line.rstrip())  
f.close()
```

1783	22	25	27	28	21	19
2214	-4	2	6	7	6	0
9934	-40	-32	-26	-21	-24	-32
5538	15	17	21	22	23	19

How to access this 6?

SPLIT STRINGS

```
# splitting strings
```

```
f = open("temptable.txt", "r")
for line in f:
    print(line.rstrip().split())

f.close()
```

split() accepts a
delimiter
(whitespace by
default)

```
['1783', '22', '25', '27', '28', '21', '19']
['2214', '-4', '2', '6', '7', '6', '0']
['9934', '-40', '-32', '-26', '-21', '-24', '-32']
['5538', '15', '17', '21', '22', '23', '19']
```

SELF-CHECK

What is the output?

```
# split  
  
path = "/usr/local/bin/python"  
parts = path.split('/')  
  
print(parts)
```



Delimiter to
determine split
points

SELF-CHECK

```
# split

path = "/usr/local/bin/python"
parts = path.split('/')

print(parts)

['', 'usr', 'local', 'bin', 'python']
```

WRITING TEXT FILES

```
f = open("file_to_write.txt", "w")
```

```
f = open("file_to_write.txt", "a")
```

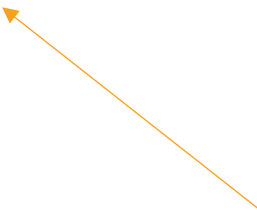
Write: creates or overwrites the contents of a file

Append: creates or appends to the contents of an existing file

WRITE EXAMPLES

```
#creates or overwrites the file  
f = open("my_file.txt", "w")  
  
f.write("test")  
  
f.close()
```

Does not automatically
add a '\n'



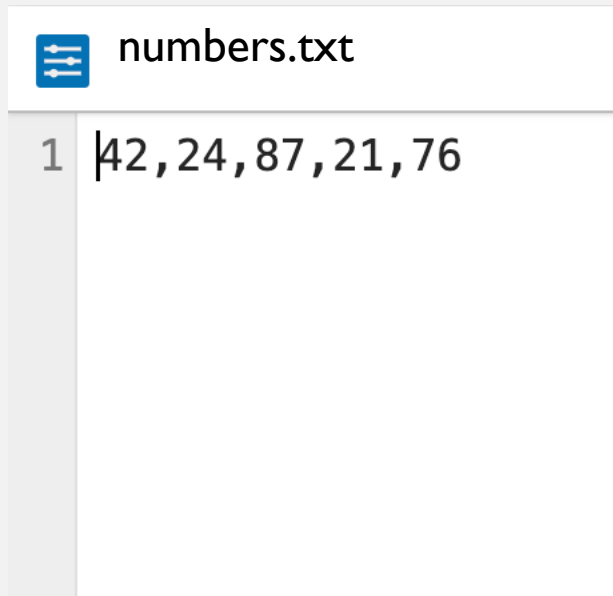
WRITE EXAMPLES

If you ran this code over and over it would append "test" to the file over and over

```
f = open("my_file.txt", "a")  
f.write("test")  
f.close()
```

WRITING

Consider we want to write a list of numbers out to file



```
numbers.txt
1 |42,24,87,21,76
```

```
numbers = [42 , 24 , 87 , 21 , 76]
```

```
f = open("numbers.txt", "w")
```

```
cnt = 0
```

```
for n in numbers:
```

```
    if cnt < len(numbers)-1:
```

```
        f.write(str(n) + ",")
```

```
    else:
```

```
        f.write(str(n))
```

```
    cnt += 1
```

```
f.close()
```

STRING .JOIN

- .join concatenates a sequence of strings together with a specified separator
- the separator is the string on which .join is called
- Example:

separator

Sequence of strings

```
phone = "-".join(["902", "555", "5555"])  
print(phone)
```

902-555-5555

WRITING TO FILE

```
numbers = [42 , 24 , 87 , 21 , 76]
```

```
f = open("numbers.txt", "w")  
f.write( ",".join([str(n) for n in numbers]) )  
f.close()
```

.join with list comprehension to convert the ints to strings



numbers.txt

```
1 |42,24,87,21,76
```

PATHS

- **Absolute paths:** specify the entire path from the beginning of the file system to a particular file

Windows example:

```
"C:\Users\Chris\My Documents\awesomedata.csv"
```

macOS example:

```
"/home/chris/Documents/awesomedata.csv"
```



At the start of the file system in the folder home then chris then Documents there is a file awesomedata.csv



- **Relative paths:** specify the path from a current working directory (example where your program is running)
- `..` Is a special name that refers to the parent directory
- `.` Is a special name that refers to the current directory

```
"../.. / experiment / data / specialdata.txt"
```

Parent directory

From our current directory go up to the parent, go up to the parent again, then enter a directory called experiment then data and find a file specialdata.txt

OS DIFFERENCES

- On windows paths are separated using a backward slash “\”
- On macOS and Linux paths are separated by a forward slash
- To avoid writing code that only works on windows or linux for example
 - Use the os module and path.join

Do not need to
worry if it is
“data/data.txt” or
“data\data.txt”

```
# path example
#open and print the lines of the file data.txt found in the data folder

import os

#relative path from the current directory
filename = os.path.join("data","data.txt")

f = open(filename)

for line in f:
    print(line.rstrip())

f.close()
```

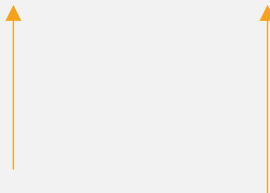
PATH .JOIN

FIND A PARTNER

- Read each line of the instructions into a tuple containing the command and the integer value e.g.:

[('FORWARD', 100), ('RIGHT', 90), ('FORWARD', 100), ('RIGHT', 90), ('FORWARD', 100), ('RIGHT', 90), ('FORWARD', 100), ('RIGHT', 90)]

command Integer value



instructions.txt

```
FORWARD 100
RIGHT 90
FORWARD 100
RIGHT 90
FORWARD 100
RIGHT 90
FORWARD 100
RIGHT 90
```

TRY AT HOME

- Count the words in the file gentle.txt found in the directory poem
 - Print out the number of words in the file
- Read the text file game_board.txt into a 2D list where each position in the 2D list is a single character
 - print out the location of the “P” (index position in the 2D list)
- Ask the user to enter their first name and then create a file called their name and the extension .txt
 - Add the length of their name to the file